

NAME: MUHAMMAD USMAN

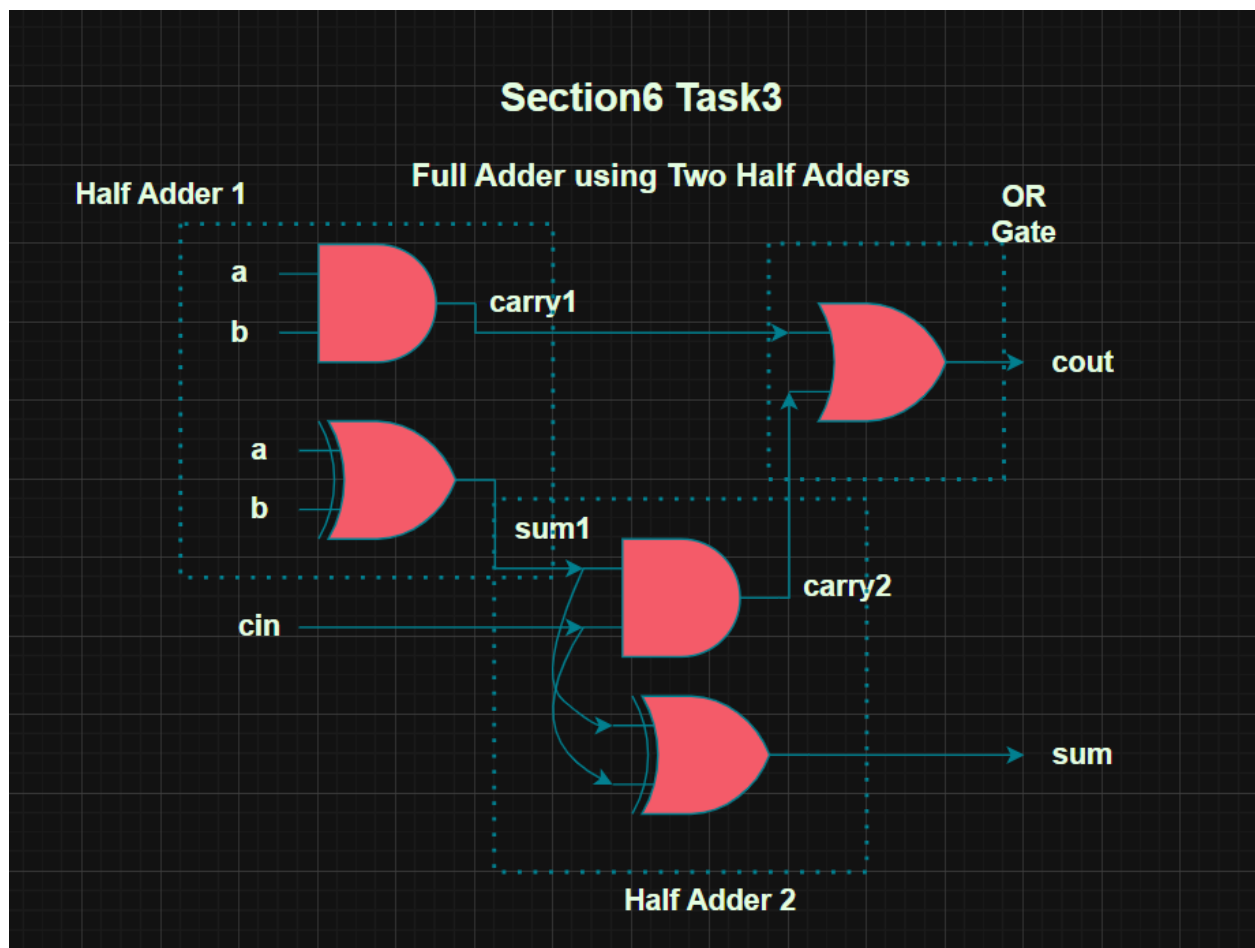
REG. NO# 2024-EE-171

1-EDA Playground link of Task3: <https://edaplayground.com/x/bhvi>

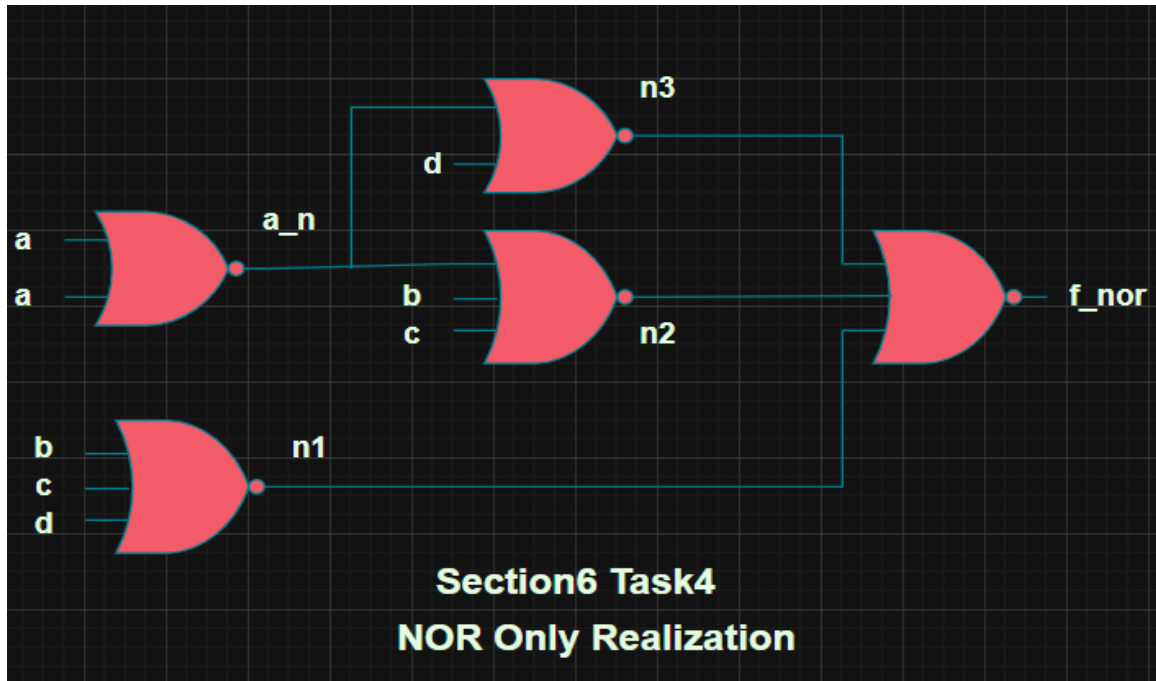
EDA Playground link of Task4: <https://edaplayground.com/x/fYSs>

EDA Playground link of Task5: <https://edaplayground.com/x/KZda>

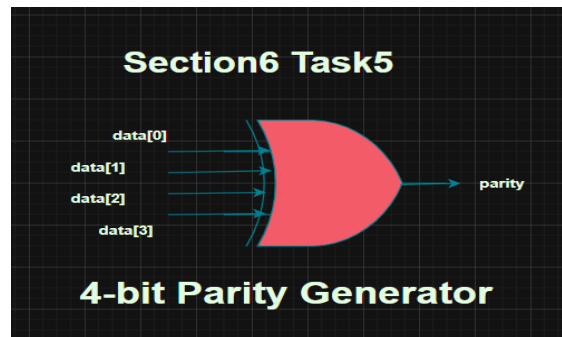
2-Schematic Diagram:



Task3_schematic_diagram.png



Task4_schematic_diagram.png



Task5_schematic_diagram.png

3-Output Screenshot:

Contains Synopsys proprietary information.

Compiler version X-2025.06-SP1_Full64; Runtime version X-2025.06-SP1_Full64; Jul 17 10:47 2026

[PASS]: a=0 b=0 | assign(sum=0, carry=0) | proc(sum=0, carry=0)

[PASS]: a=0 b=1 | assign(sum=1, carry=0) | proc(sum=1, carry=0)

[PASS]: a=1 b=0 | assign(sum=1, carry=0) | proc(sum=1, carry=0)

[PASS]: a=1 b=1 | assign(sum=0, carry=1) | proc(sum=0, carry=1)

\$stop at time 40 Scope: tb File: testbench.sv Line: 25

uccli% Finding VCD file...

./dump.vcd

[2026-07-17 14:47:59 UTC] Opening EPWave...

Done

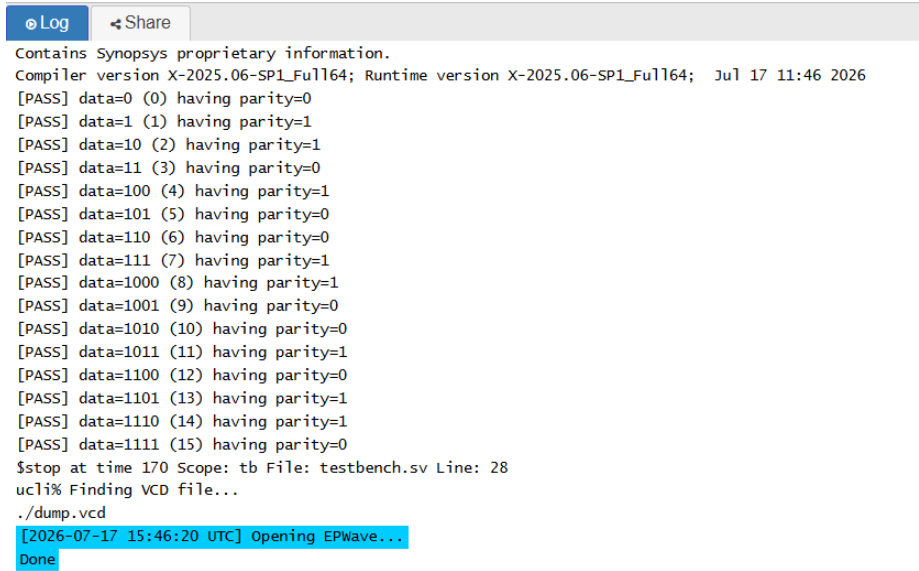
Task3_ha_log_result.png

```
Contains Synopsys proprietary information.
Compiler version X-2025.06-SP1_Full164; Runtime version X-2025.06-SP1_Full164; Jul 17 12:09 2026
[PASS] HA: a=0 b=0 | assign(sum=0, carry=0) | proc(sum=0, carry=0)
[PASS] HA: a=0 b=1 | assign(sum=1, carry=0) | proc(sum=1, carry=0)
[PASS] HA: a=1 b=0 | assign(sum=1, carry=0) | proc(sum=1, carry=0)
[PASS] HA: a=1 b=1 | assign(sum=0, carry=1) | proc(sum=0, carry=1)
[PASS] FA: a=0 b=0 cin=0 having sum=0 cout=0
[PASS] FA: a=0 b=0 cin=1 having sum=1 cout=0
[PASS] FA: a=0 b=1 cin=0 having sum=1 cout=0
[PASS] FA: a=0 b=1 cin=1 having sum=0 cout=1
[PASS] FA: a=1 b=0 cin=0 having sum=1 cout=0
[PASS] FA: a=1 b=0 cin=1 having sum=0 cout=1
[PASS] FA: a=1 b=1 cin=0 having sum=0 cout=1
[PASS] FA: a=1 b=1 cin=1 having sum=1 cout=1
$stop at time 120 Scope: tb File: testbench.sv Line: 46
ucli% Finding VCD file...
./dump.vcd
[2026-07-17 16:09:42 UTC] Opening EPWave...
Done
```

Task3_fa_log_result.png

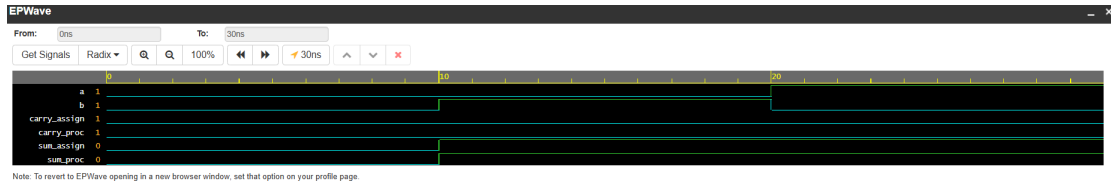
```
Log Share
Contains Synopsys proprietary information.
Compiler version X-2025.06-SP1_Full164; Runtime version X-2025.06-SP1_Full164; Jul 17 11:29 2026
[PASS]: a=0 b=0 c=0 d=0 | direct=0 | nor=0
[PASS]: a=0 b=0 c=0 d=1 | direct=1 | nor=1
[PASS]: a=0 b=0 c=1 d=0 | direct=1 | nor=1
[PASS]: a=0 b=0 c=1 d=1 | direct=1 | nor=1
[PASS]: a=0 b=1 c=0 d=0 | direct=1 | nor=1
[PASS]: a=0 b=1 c=0 d=1 | direct=1 | nor=1
[PASS]: a=0 b=1 c=1 d=0 | direct=1 | nor=1
[PASS]: a=0 b=1 c=1 d=1 | direct=1 | nor=1
[PASS]: a=1 b=0 c=0 d=0 | direct=0 | nor=0
[PASS]: a=1 b=0 c=0 d=1 | direct=0 | nor=0
[PASS]: a=1 b=0 c=1 d=0 | direct=0 | nor=0
[PASS]: a=1 b=0 c=1 d=1 | direct=1 | nor=1
[PASS]: a=1 b=1 c=0 d=0 | direct=0 | nor=0
[PASS]: a=1 b=1 c=0 d=1 | direct=1 | nor=1
[PASS]: a=1 b=1 c=1 d=0 | direct=0 | nor=0
[PASS]: a=1 b=1 c=1 d=1 | direct=1 | nor=1
$stop at time 160 Scope: tb File: testbench.sv Line: 21
ucli% Finding VCD file...
./dump.vcd
[2026-07-17 15:29:18 UTC] Opening EPWave...
Done
```

Task4_log_result.png

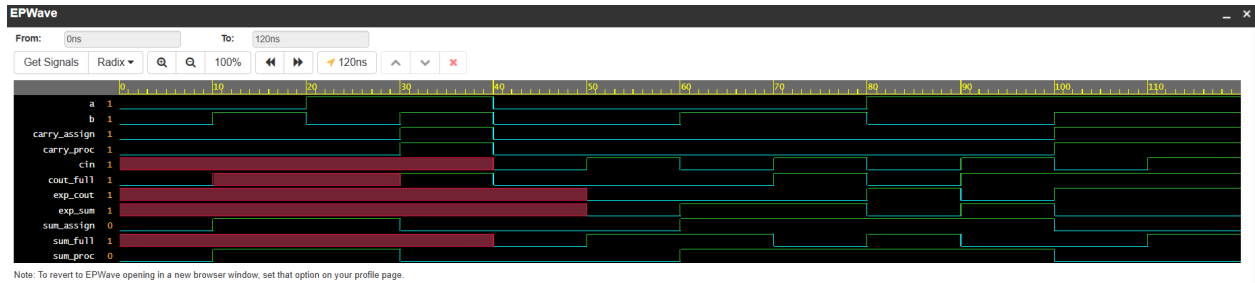


Task5_log_result.png

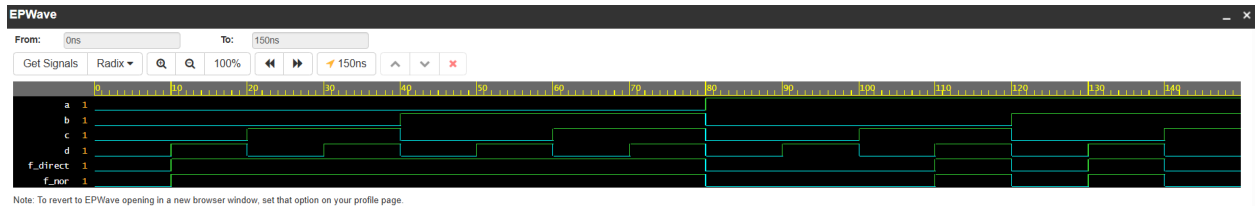
4-Waveform Screenshot:



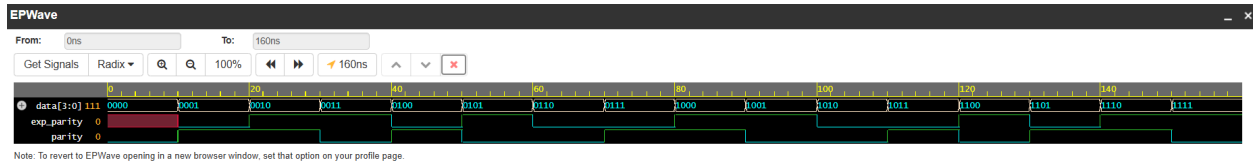
Task3_ha_waveform.png



Task3_fa_waveform.png



Task4_waveform.png



Task5_waveform.png

5-Explanation:

i-Task3 Reasoning:

In my point of view, the assign version and the always_comb version synthesize to the same gates even though they are written differently because when we synthesize our design in any software it will automatically synthesize our design in such a way that it will be the optimized version of our design, so, it will synthesize to the same gates as the original idea is same.

ii-Task4 Boolean Algebra / De Morgan Steps:

We start with:

$$f = (b + c + d) \cdot (a' + b + c) \cdot (a' + d)$$

- **Step 1 – Identify the three OR terms (literals):**

Let:

$$X = (b + c + d)$$

$$Y = (a' + b + c)$$

$$Z = (a' + d)$$

Now $f = X \cdot Y \cdot Z$.

- **Step 2 – Apply De Morgan's theorem to the whole AND:**

$$\text{De Morgan says: } X \cdot Y \cdot Z = (X' + Y' + Z')$$

So:

$$f = ((b+c+d)' + (a'+b+c)' + (a'+d)')$$

- **Step 3 – Interpret the terms as NOR outputs:**

- $(b+c+d)'$ is a 3-input NOR of b, c, d

- $(a'+b+c)'$ is a 3-input NOR of a', b, c

- $(a'+d)'$ is a 2-input NOR of a' and d

(But we need a' first. We can get a' using a NOR gate with both inputs tied to a , i.e. $a' = (a+a)' = \text{NOR}(a,a)$)

- **Step 4 – Final stage:**

The outer complement $(\dots)'$ is a 3-input NOR that combines the three intermediate results.

Resulting all-NOR network:

$a_n = \text{NOR}(a, a) \rightarrow$ this gives a'

$n1 = \text{NOR}(b, c, d) \rightarrow$ gives $(b+c+d)'$

$n2 = \text{NOR}(a_n, b, c) \rightarrow$ gives $(a'+b+c)'$

$n3 = \text{NOR}(a_n, d) \rightarrow$ gives $(a'+d)'$

$f = \text{NOR}(n1, n2, n3) \rightarrow$ final output